LegalBuddy — Challenging DevOps & MLOps Grilling Playbook

High-Stakes Architectural Defense, Edge-Case Failure Scenarios & Live Production Incident Triage

Format: Senior / Staff / Lead DevOps Interview
Level: Extreme Technical Depth & Trade-offs

Focus Areas: Kernel OOMs, Swarm vs K8s, Swap Thrashing
Tools Tested: Docker Swarm, Traefik, azcopy, vLLM, Qdrant

Evaluation: System Defense, Root-Cause Analysis, Hardware Math

How to Use This Grilling Playbook:

Interviews for senior DevOps and MLOps positions do not test whether you know what Docker is—they probe for the **breaking points** of your architecture. Interviewers will push you with questions like: *"What happens when your 64GB swap file thrashes and locks the kernel?", "Why didn't you use Triton Inference Server?", "How did your dual-network design prevent cross-tenant request forgery?", "Calculate the exact VRAM footprint of a 31B model with KV cache."*
For each question, study the **Interviewer's Trap**, note the **Junior Flawed Answer** to avoid, and master the **Senior/Staff Defense**.

Category 1: Architectural Defense & Trade-Offs

DIFFICULTY: HARD ARCHITECTURE DEFENSE

Q1: "You used Dokploy and Docker Swarm instead of Kubernetes. Isn't that an amateur choice for an enterprise legal application?"

THE TRAP: TESTING WHETHER YOU ARE DOGMATIC ABOUT BUZZWORDS OR UNDERSTAND PRAGMATIC ROI, RESOURCE BUDGETS, AND OPERATIONAL COMPLEXITY.

> **Junior Answer:** "Kubernetes was too hard to learn and we just wanted something fast, so we picked Dokploy." *(Shows lack of senior judgment and admits technical inadequacy.)*

Staff DevOps Defense:
"Choosing an orchestration platform is an exercise in resource budgeting and failure domain management. Our production infrastructure runs on a single node with 4 vCPUs and 7.8 GB of RAM.

If we deployed a production Kubernetes control plane (even lightweight distributions like K3s with etcd), the control plane alone consumes 1.5 to 2 GB of resident memory and continuous background CPU cycles for reconciliation loops and pod lifecycle monitoring. On an 8GB machine where our application's resident 768-dimensional multilingual embedding model requires ~2.2 GB of RAM, allocating 25% of our host memory to Kubernetes infrastructure would severely starve our application and trigger kernel OOM panics.

Dokploy operates natively on Docker Swarm. It gives us the exact primitives we actually need: declarative compose stacks, healthcheck-gated zero-downtime rolling updates, GitOps webhook automation, and built-in Traefik reverse proxy with Let's Encrypt TLS renewal—all with an operational memory overhead under 250 MB. When our traffic scales beyond a single node, Docker Swarm scales out to multi-node overlay networks with zero changes to our compose manifest, or we can graduate to managed Kubernetes (EKS/AKS) without redesigning our container images."

DIFFICULTY: EXPERT NETWORKING & SECURITY

Q2: "In your dual-network setup, Traefik, Next.js, and Qdrant all sit on dokploy-network. What prevents an attacker who compromises Next.js from querying Qdrant directly?"

THE TRAP: PROBING YOUR EXACT CONTAINER NETWORK BOUNDARIES, DOCKER OVERLAY ROUTING, AND APPLICATION-LAYER AUTHENTICATION DEFENSES.

> **Junior Answer:** "The Next.js container only has frontend code so it can't talk to Qdrant." *(Factually false—any container on the same Docker bridge/overlay can initiate TCP handshakes to any other container IP.)*

Staff DevOps Defense:

"In our architecture, defense-in-depth is enforced at three distinct layers:

- **Network Segmentation:** In `docker-compose.prod.yml`, the Next.js service is intentionally configured to proxy chat requests strictly to the FastAPI backend over the isolated, project-scoped `internal` network via `API_URL=http://api:8000`. Next.js does not contain Qdrant client libraries or connection drivers.
- **Mandatory Authentication Token:** Qdrant is deployed with mandatory API key authentication (`QDRANT_API_KEY`). This secret is strictly injected into the FastAPI container's environment via Dokploy and is **never** shared or exposed to the `web` container or frontend client bundles. Even if an attacker obtained remote code execution inside the Next.js container, unauthenticated TCP requests to `qdrant:6333` are rejected with HTTP 401 Unauthorized.
- **Container Hardening:** Next.js runs as an unprivileged user (`nextjs:nodejs`, UID 1001) in a read-only minimal container without curl, netcat, or package managers, severely constraining lateral movement. In an enterprise VPC upgrade, we would enforce Calico or Cilium NetworkPolicies to strictly deny egress traffic from the `web` pod CIDR to port 6333 at the kernel packet filtering level."

DIFFICULTY: HARD DEPLOYMENT LIFECYCLE

Q3: "If Uvicorn crashes in production, what happens to inflight requests, and how does Dokploy prevent an infinite restart loop?"

THE TRAP: TESTING UNDERSTANDING OF DOCKER RESTART POLICIES, SWARM ROLLBACK MECHANISMS, AND HTTP CONNECTION DRAINING.

> **Junior Answer:** "Docker just restarts it automatically because restart is set to always." *(Misses what happens to inflight sockets and ignores how to break restart loops.)*

Staff DevOps Defense:

"Our production stack configures `restart: unless-stopped` and is orchestrated via Docker Swarm service specs.

When Uvicorn crashes unexpectedly:

- **Inflight Connections:** Any HTTP socket currently processing a request receives a TCP `RST` or connection reset. Traefik detects the sudden backend socket termination and responds to the client with an immediate HTTP 502 Bad Gateway.
- **Healthcheck Invalidation:** Docker daemon immediately marks the container task dead and spawns a replacement container. However, because our startup gate requires `get_embedding_model()` and `verify_qdrant()` to complete before `/rag/health` returns 200, Traefik's dynamic load balancer will NOT route any incoming traffic to the new container until its healthcheck passes.
- **Crash Loop Defense:** If the crash is due to a permanent defect (e.g. corrupted Qdrant collection or missing env var), the container fails startup during lifespan, exiting with code 1. In Dokploy's Swarm deployment spec, we define maximum retry attempts (`max_attempts: 3`) and failure action (`rollback`). Once retries are exhausted, Swarm freezes the service in a failed state rather than thrashing CPU cycles, and alerts are emitted via Dokploy's webhook notifications."

Category 2: Live Production Incident Triage (War Room Scenarios)

DIFFICULTY: EXPERT WAR ROOM INCIDENT

Q4: "It's 2:00 AM. Traefik is serving 502 Bad Gateway. You SSH into tommy-vm and run 'docker ps'. Both 'web' and 'api' are reported as running. Walk me through your exact command-line triage sequence."

THE TRAP: CHECKING IF YOU PANIC, GUESS RANDOMLY, OR EXECUTE A DISCIPLINED LAYER-BY-LAYER ROOT CAUSE TRIAGE (L4/L7 NETWORKING -> CONTAINER HEALTH -> APPLICATION LOGS -> HOST RESOURCES).

Senior DevOps Incident Playbook:
"I follow a disciplined top-down triage protocol:

1. **Inspect Container Health States:**
   `docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"`
   Analysis: 'Running' does not mean 'healthy'. If `api` says `(unhealthy)`, Traefik or Next.js has stopped routing to it.
2. **Direct HTTP Probe of the Internal Health Endpoint:**
   Bypass Traefik and test the API directly from the host virtual network:
   `docker exec -it $(docker ps -qf name=api) python -c "import urllib.request; print(urllib.request.urlopen('http://localhost:8000/rag/health', timeout=3).read())"`
   Branch A: If it returns `{"status":"ok"}`, the API is healthy. The failure is in network resolution or Traefik routing.
   Branch B: If it hangs or raises ConnectionRefused, Uvicorn's event loop is blocked or crashed.
3. **Check Host Resource Starvation (OOM / CPU Saturation):**
   `free -h && vmstat 1 5 && dmesg -T | grep -i oom`
   Analysis: Verify if the Linux kernel OOM-killer killed a child worker process or if swap thrashing caused thread starvation.
4. **Tail Real-Time Logs for Deadlocks or Upstream Timeouts:**
   `docker logs --tail 100 --timestamps $(docker ps -qf name=api)`
   `docker logs --tail 100 --timestamps $(docker ps -qf name=web)`
   What I'm looking for: Is Qdrant timing out? Did an external LLM call (Gemini/Groq) hang without a timeout, locking Uvicorn worker threads? Did Uvicorn hit an unhandled exception during lifespan?
5. **Mitigate & Restore:** If the API is deadlocked, execute a clean rolling restart: `docker service update --force legalbuddy_api`. Once restored, capture logs and dump state for post-mortem analysis."

DIFFICULTY: EXPERT WAR ROOM INCIDENT

Q5: "During GGUF conversion of Gemma-4-31B on Azure, the SSH session hangs, CPU drops to 0%, and the VM becomes totally unresponsive. What is happening inside the Linux kernel, and how do you architecturally prevent it?"

THE TRAP: TESTING DEEP UNDERSTANDING OF LINUX VIRTUAL MEMORY MANAGEMENT (VMM), SWAP THRASHING, ANONYMOUS PAGE EVICTION, AND DISK I/O QUEUING.

> **Junior Answer:** "The VM ran out of memory, so I would just restart the VM and use a bigger instance." *(Doesn't explain why SSH froze or the kernel mechanism.)*

Staff DevOps Defense:
"This is classic **Swap Thrashing / Kswapd Lockup**:

- **Kernel Mechanism:** When `convert_hf_to_gguf.py` parses 31 billion weights (~62GB uncompressed), memory allocations exceed physical RAM. The Linux kernel's page allocator invokes `kswapd` to aggressively page out anonymous memory to our 64GB swap file. As working set memory exceeds physical RAM, the CPU spends 99% of its cycles waiting on synchronous NVMe disk I/O (`iowait`) to swap pages back and forth rather than executing instructions.
- **Why SSH Froze:** The SSH daemon (`sshd`) and user shell processes had their text and stack pages evicted to swap. When incoming TCP packets arrived, the kernel had to swap them back in from a saturated disk I/O queue, causing SSH to hang and look completely dead.
- **Architectural Prevention & Remediation:**
  1. **Tune Linux Kernel Swappiness:** Lower `vm.swappiness` from default 60 to 10 (`sudo sysctl vm.swappiness=10`). This instructs the kernel to prefer evicting page cache rather than swapping active application heap pages.
  2. **Tune Dirty Page Ratios:** Set `vm.dirty_ratio=10` and `vm.dirty_background_ratio=5` to flush I/O continuously in smaller chunks rather than blocking the I/O bus with massive 10GB write bursts.
  3. **Storage QoS:** Place the swap file on an ephemeral NVMe data disk (`/dev/sdc` formatted as XFS) with striped read/write IOPS, completely separate from the OS disk (`/dev/sda`). This ensures system daemons and SSH remain responsive even during heavy swap operations.
  4. **Chunked Processing:** Use `--outtype f16` streaming conversion flags in `convert_hf_to_gguf.py` to process tensor tensors incrementally rather than loading all shard safetensors simultaneously into memory."

DIFFICULTY: HARD CI/CD & BUILD DRIFT

Q6: "A junior developer adds a new package to pyproject.toml. CI passes, but production crashes with 'CUDA error: out of memory' on boot. How did this happen if you pinned CPU PyTorch, and how do you enforce a strict lock?"

THE TRAP: TESTING UNDERSTANDING OF UV LOCKFILE RESOLUTION, DEPENDENCY TREE CONFLICTS, AND CI GATE ENFORCEMENT.

Senior DevOps Answer:
"This occurs when a developer adds a package that depends on `torch` (like `torchvision` or `torchaudio`) and runs `uv add <package>` without specifying the custom index, or updates dependencies without the `--frozen` flag.

If the package requests a PyPI wheel that doesn't respect our `pytorch-cpu` source, Astral `uv` might resolve the dependency to the default PyPI index, pulling down standard CUDA-enabled wheels.

How We Enforce Strict Immutability:

- **Index Isolation (`explicit = true`):** In `pyproject.toml`, the `pytorch-cpu` index has `explicit = true`. This forbids `uv` from using it as a general fallback, forcing explicit source mapping.
- **CI Enforcement Gate:** In `.github/workflows/ci.yml`, the CI job runs `uv sync --frozen --check`. The `--check` flag asserts that `uv.lock` matches `pyproject.toml` exactly. If any dependency is out of sync or unlocked, CI aborts immediately.
- **Lockfile Inspection Step:** We add an automated CI check:
  `uv run python -c "import torch; assert not torch.cuda.is_available(); assert '+cpu' in torch.__version__"`
  If any CUDA dynamic library is bundled, the assertion raises an error and blocks the merge before Dokploy ever receives a webhook."

Category 3: MLOps, Hardware Math & Azure Model Serving

DIFFICULTY: EXPERT HARDWARE MATH

Q7: "Prove to me with exact numbers why Gemma-2-27B requires ~54GB in FP16, and calculate the exact VRAM needed for Q4_K_M quantization with a 4,096 context window."

THE TRAP: INTERVIEWERS HATE HAND-WAVING. THEY WANT EXACT MATHEMATICAL DERIVATIONS OF MODEL PARAMETERS, BYTE PRECISION, KV CACHE SIZE, AND ACTIVATION OVERHEAD.

Staff MLOps Mathematical Derivation:
"Here is the precise hardware calculation:

1. **Model Weight Memory (FP16 / 16-bit):**
   • Parameters (N): 27×10^9 parameters (27 billion).
   • Precision: FP16 uses 2 bytes per parameter (16 bits / 8 bits = 2 bytes).
   • Base Weight VRAM: $27 \times 10^9 \times 2 \text{ bytes} = 54 \times 10^9 \text{ bytes} \approx \mathbf{50.3\text{ GiB}}$ (or 54 GB in decimal).
   • Plus model architecture overhead (embeddings, layer norms, unembedding projection): $\approx \mathbf{54\text{ GB}}$.
2. **Quantization to GGUF Q4_K_M (4-bit medium):**
   • Q4_K_M uses 4.5 bits on average per parameter (combining 4-bit blocks with 6-bit quantization scales for sensitive attention layers).
   • Compressed Weights: $27 \times 10^9 \times (4.5 / 8) \text{ bytes} \approx \mathbf{15.18\text{ GB}}$ footprint on disk and VRAM.
3. **KV Cache Memory at 4,096 Context Length:**
   • Formula: $\text{KV Cache Size} = 2 \times (\text{layers}) \times (\text{key-value heads}) \times (\text{head dimension}) \times (\text{context length}) \times (\text{bytes per element})$.
   • For Gemma-2-27B: 46 layers, 16 KV heads (Grouped Query Attention), 128 head dim, FP16 precision (2 bytes).
   • $\text{KV Cache} = 2 \times 46 \times 16 \times 128 \times 4,096 \times 2 \text{ bytes} \approx \mathbf{1.54\text{ GB}}$ per concurrent request.
4. **Total Inference Footprint:**
   • Weights (15.2 GB) + KV Cache for batch 1 (1.5 GB) + Activation overhead (0.5 GB) $\approx \mathbf{17.2\text{ GB}}$.
   • By setting `--gpu-memory-utilization 0.9` and running `llama-server` with offloaded context, it fits cleanly on an affordable 16GB T4/L4 GPU VM (using minimal CPU offloading for excess KV cache), eliminating the need for an expensive 80GB A100."

DIFFICULTY: HARD MLOPS ARCHITECTURE

Q8: "Why did you use llama-server and vLLM instead of Triton Inference Server or Ollama for serving in production?"

THE TRAP: TESTING YOUR COMPARATIVE EVALUATION OF INFERENCE ENGINES ACROSS THROUGHPUT, PAGED ATTENTION, OPERATIONAL OVERHEAD, AND PRODUCTION READINESS.

Senior MLOps Answer:

"We benchmarked these tools against our deployment targets:

- **Why NOT Ollama?** Ollama is fantastic for local desktop developer workflows, but it is not built for cloud production deployments. It bundles its own daemon, lacks fine-grained batch scheduling controls, hides low-level KV cache allocation parameters, and introduces extra abstraction layers that make container orchestration difficult.
- **Why NOT Triton Inference Server?** NVIDIA Triton is an enterprise multi-model server, but setting up Triton with the TensorRT-LLM backend requires heavy model compilation for specific GPU compute capabilities (e.g. compiling distinct engines for Ampere vs Hopper vs Ada), producing rigid engine artifacts that cannot easily migrate between cloud VM types. The operational burden and build times (30+ minutes per engine) were disproportionate.
- **Why vLLM & llama.cpp?**
  - **vLLM:** Industry gold standard for high-concurrency serving. Its *PagedAttention* algorithm manages KV cache like virtual memory paging in an OS kernel, achieving 2–4x higher throughput under multi-user concurrency with dynamic continuous batching.
  - **llama.cpp (llama-server):** Zero-dependency C/C++ binary with negligible memory overhead. It supports GGUF Q4_K_M quantization natively and exposes an OpenAI-compatible `/v1/chat/completions` endpoint with built-in streaming (SSE), allowing us to hot-swap models without changing any client-side code in FastAPI."

Category 4: Vector Database & Data Pipeline Resilience

DIFFICULTY: EXPERT ZERO-DOWNTIME MIGRATION

Q9: "When you upgraded from triBne-e5-small (384-dim) to multilingual-e5-base (768-dim), how did you execute the collection recreation without causing downtime for users?"

THE TRAP: TESTING BLUE/GREEN DATABASE MIGRATION STRATEGIES, VECTOR INDEX REBUILD ZERO-DOWNTIME PATTERNS, AND CONFIGURATION MANAGEMENT.

> **Junior Answer:** "We ran `uv run ingest`, which dropped the collection and re-created it." *(During ingestion, which takes 30-45 minutes, every user query fails with 500 CollectionNotFound!)*

Staff DevOps Migration Strategy:

"In vector search, changing embedding dimensions from 384 to 768 is a breaking schema migration—a collection cannot store mixed vector dimensions.

We executed a **Blue/Green Collection Alias Migration**:

1. **Ingest to Shadow Collection (Green):** Ingested the 35,000+ sections into a brand-new collection named `legal_acts_v2_768` using the 768-dim model. The live production API continued querying the active collection (`legal_acts_v1_384`) without any interruption.
2. **Readiness & Verification Gate:** Ran our evaluation harness (`eval/run_eval.py --collection legal_acts_v2_768`) against the new collection to verify recall metrics and ensure all payload indexes (`act_year`, `section_uid`) were fully built.
3. **Atomic Alias Swap:** In Qdrant, we used the collection aliases API to atomically re-point the production alias:

```
client.update_collection_aliases(
    change_aliases_operations=[
        models.CreateAliasOperation(
            collection_name="legal_acts_v2_768", alias_name="legal_acts_event_rag_full"
        )
    ]
)
```

   This alias update is an atomic in-memory pointer swap in Qdrant—zero dropped packets, zero 500 errors.
4. **Configuration Update & Cleanup:** Updated `EMBEDDING_MODEL` in Dokploy to `intfloat/multilingual-e5-base` and deployed the API. Once stable, dropped the legacy `legal_acts_v1_384` collection to reclaim RAM."

DIFFICULTY: HARD RESILIENCE & FAILURES

Q10: "During bulk ingestion of 35,000 legal sections, Qdrant crashes with an OOM kill. How do you tune Qdrant's HNSW indexing to survive on constrained hardware?"

THE TRAP: TESTING VECTOR DATABASE INTERNALS, HNSW GRAPH PARAMETERS, MEMTABLE FLUSH RATES, AND DISK PAYLOAD STORAGE.

Senior DevOps Answer:
"When Qdrant ingests high-dimensional vectors (768 floats $\times$ 4 bytes = ~3KB per vector plus metadata), memory spikes are caused by simultaneous **HNSW graph construction** and in-memory vector indexing.

To guarantee ingestion stability on constrained hardware:

- **Disable Indexing During Ingestion:** In Qdrant's collection configuration, set `indexing_threshold: 0` during bulk upsert. This instructs Qdrant to append points directly to WAL (Write-Ahead-Log) segments on disk without continuously building the HNSW graph in RAM. Once ingestion completes, restore `indexing_threshold: 20000` to build the graph in a single pass.
- **On-Disk Payloads & Vectors:** Configure `on_disk_payload: true` and `on_disk: true` for vectors in `HnswConfig`. This stores the vector data in memory-mapped files on disk rather than holding the entire index in physical RAM.
- **Batch Size & Concurrency Throttling:** In `apps/ingestion/pipeline.py`, we throttled batch upserts to 64 points per batch and implemented exponential backoff retries (`upsert_with_retry` with 4 attempts and `sleep(2 * attempt)`). This flushes memory buffers consistently and prevents thread exhaustion."

Category 5: Linux Internals, Security & Scale-Out System Design

DIFFICULTY: HARD CONTAINER SECURITY

Q11: "Why is running as non-root critical in our Dockerfiles, and what specific Linux capabilities did you drop or preserve?"

THE TRAP: TESTING UNDERSTANDING OF LINUX NAMESPACES, CGROUPS, UID MAPPING, AND CONTAINER BREAKOUT VULNERABILITIES.

Senior DevOps Answer:
"In Docker, the `root` user inside a container has UID 0—which maps directly to UID 0 (root) on the host Linux kernel unless user namespaces (`userns-remap`) are explicitly enabled. If an attacker exploits a remote code execution vulnerability (e.g. through a Python deserialization bug or library vulnerability) in a container running as root, any container breakout flaw (such as kernel cgroup escapes or mounted socket access) grants them full root privileges over the host operating system.

In our Dockerfiles:

- In `apps/api/Dockerfile`, we create system user `app` and switch to `USER app`.
- In `apps/web/Dockerfile`, we create group `nodejs (1001)` and user `nextjs (1001)` and switch to `USER nextjs`.
- In production Swarm configs, we drop all default Linux capabilities and retain only those strictly required for unprivileged networking:
  `cap_drop: [ALL]`, granting zero raw socket manipulation, zero module loading, and zero mount capabilities."

DIFFICULTY: EXPERT SCALE-OUT SYSTEM DESIGN

Q12: "Design an enterprise architecture for LegalBuddy to scale from our single 8GB VM to handling 10 million legal queries per day across Bangladesh with sub-200ms latency. Keep infrastructure costs under $1,500/month."

THE TRAP: TESTING YOUR ABILITY TO TRANSITION FROM A PROTOTYPE/MVP ARCHITECTURE TO A PRODUCTION MULTI-TIER, DISTRIBUTED, AUTO-SCALING CLOUD ARCHITECTURE WITH COST OPTIMIZATION.

Staff Infrastructure Architecture Blueprint:

"To handle 10M queries/day ($\approx 115 \text{ queries/sec}$ average, $\sim 350\text{ QPS}$ peak) at sub-200ms within a $1,500/month budget:

1. **Tier 1: Global Edge & Semantic Caching:**
   • Place Cloudflare CDN in front of Next.js to cache static assets and common legal queries at the edge.
   • Implement a **Redis Semantic Cache** (Upstash / Redis Cloud): 30–40% of public legal queries ('punishment for theft', 'section 302 bail rules') are identical. Hashing query embeddings and matching with cosine similarity ≥ 0.98 returns cached responses in <15ms, bypassing LLM generation and vector search completely.
2. **Tier 2: Stateless Compute (Next.js & FastAPI):**
   • Deploy Next.js and FastAPI on **Azure Container Apps (ACA)** or AWS ECS Fargate with KEDA auto-scaling based on incoming HTTP request rates ($2 \rightarrow 12$ replicas during business hours).
   • Decouple the embedding model from FastAPI into an independent microservice or utilize a dedicated CPU/GPU Triton instance with dynamic request batching.
3. **Tier 3: Clustered Qdrant Vector Engine:**
   • Move Qdrant to a 2-node replicated cluster with memory-mapped vector storage on NVMe SSDs. With payload indexes on `act_year` and `section_uid`, filtered vector search takes <10ms.
4. **Tier 4: Hybrid LLM Routing:**
   • **Primary Inference:** Self-hosted quantized Gemma-4-31B Q4_K_M on 2x Azure Spot GPU instances (e.g. `NC4as_T4_v3` or L4) behind an internal load balancer (~$450/month on Spot).
   • **Burst / Overflow Fallback:** If concurrency spikes and GPU queue latency exceeds 400ms, dynamically route excess traffic to Groq API (Llama-3.3-70B at $0.59/1M tokens) or Gemini Flash.
5. **Cost Breakdown:** Edge & Redis ($80) + Container Apps ($220) + Qdrant Cluster ($300) + 2x Spot GPU VMs ($500) + Cloudflare & Monitoring ($100) $\approx \mathbf{\$1,200/\text{month}}$ total."